International Cybersecurity & Digital Forensics Academy

Fellowship in Cybersecurity and Digital Forensics

Bash Scripting - Build a Reconnaissance Tool

WADF104 - Cybersecurity Foundations

Mr. Aminu Idris, AMCPN

6th September, 2026

Student identification

Full name	**WISDOM BINICHIUKU ODILI**
Registration number	**2026/FCDF/17059**
Linux distribution / version	**KALI LINUX / 2026.2**
VM hostname	**KALI**

Lab 3: Bash Scripting - Build a Reconnaissance Tool

Mandatory guided automation practical using Nmap, WhatWeb and DIRB

Authorisation boundary: Your script must be used only against instructor-authorised lab targets. Automation makes repeated scanning easier, so authorisation is even more important.

Lab 3 combines selected skills from the first two labs. Students are NOT expected to invent the program from nothing. Build it stage by stage, type each code block, test each stage, and explain what it does.

This Lab is a guided Bash Scripting automation practical using Nmap, WhatWeb and DIRB. It Prompts for a target IP/domain, display a menu: WhatWeb, Nmap, DIRB, runs the selected tool against the user-supplied target and displays results.

- Prompt for a target IP/domain.
- Display a menu: WhatWeb, Nmap, DIRB.
- Use Bash conditionals (case/if).
- Run the selected tool against the user-supplied target.
- Display output clearly.
- No hardcoded target.
- Script must be executable.
- Show successful execution of at least two different tools.

Assessment Requirements

- Prompt for a target IP/domain.
- Display a menu: WhatWeb, Nmap, DIRB.
- Use Bash conditionals (case/if).
- Run the selected tool against the user-supplied target.
- Display output clearly.
- No hardcoded target.
- Script must be executable.
- Show successful execution of at least two different tools.

Part 1 - Bash Building Blocks

Concept	Meaning
Shebang	Chooses the interpreter.
Variable	Stores a value.
read	Accepts keyboard input.
echo	Displays text.
if	Runs code when a condition is true.

case	Chooses one of several menu branches.
$name	Reads a variable value.
chmod +x	Adds executable permission.
tee	Shows output and saves it to a file.

Part 2 - Create the Project

Step 1: Make a folder

Objective: Keep your lab files organised.

```
mkdir -p ~/lab3-recon
cd ~/lab3-recon
pwd
```

What this means: Creates and enters the working folder.

What you should expect: Path ends in /lab3-recon.

Record in your report: Screenshot the path.

Step 2: Check tools

Objective: Confirm required commands exist.

```
command -v bash
command -v nmap
command -v whatweb
command -v dirb
```

What this means: command -v prints a program path when available.

What you should expect: Paths for each installed tool.

Record in your report: Record missing tools and tell the instructor.

Step 3: Create script

Objective: Open a new file.

```
nano recon_tool.sh
```

What this means: Nano is a terminal editor.

What you should expect: Blank file opens.

Record in your report: Screenshot before coding.

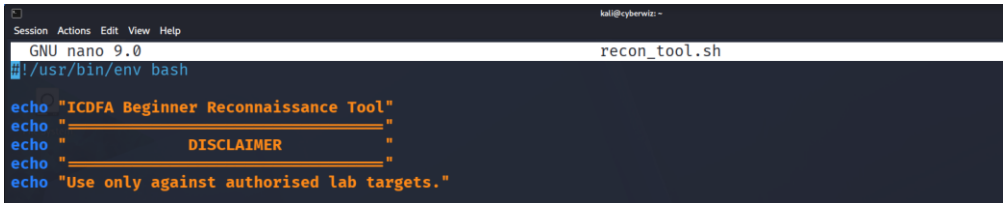
Part 3 - Build the Script in Stages

Stage A: Header

```
#!/usr/bin/env bash

echo "ICDFA Beginner Reconnaissance Tool"
echo "----------------------------------"
echo "Use only against authorised lab targets."
```

- The shebang tells Linux to run the file with Bash.
- echo prints the title and warning.



Stage B: Read the target

```
echo
read -rp "Enter authorised target IP address or domain: " target

echo "Target entered: $target"
```

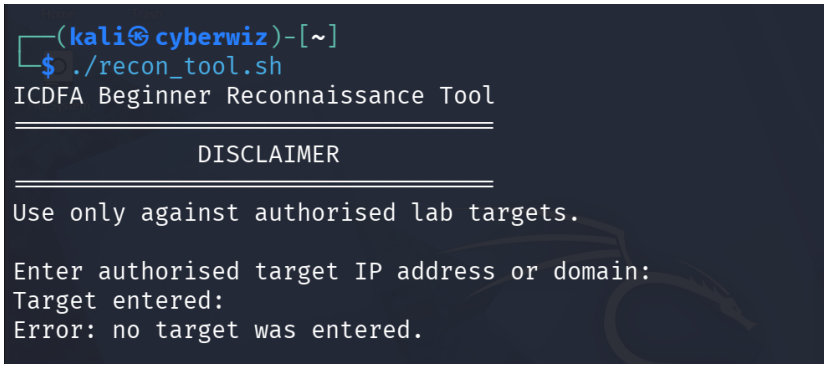
- read accepts input.
- -r keeps backslashes literal.
- -p displays the prompt.
- target is the variable name.
- $target reads the stored value.

Stage C: Validate empty input

```
if [[ -z "$target" ]]; then
   echo "Error: no target was entered."
   exit 1
fi
```

Run the script once and press Enter without a target. It must stop with the error message.

Validating empty input:



Stage D: Menu

```
echo
echo "Select a reconnaissance tool:"
echo "1) WhatWeb"
echo "2) Nmap"
echo "3) DIRB"
echo "4) Exit"
read -rp "Enter your choice [1-4]: " choice
```

Stage E: Understand case

```
case "$choice" in
    1) echo "Option 1" ;;
    2) echo "Option 2" ;;
    *) echo "Invalid option" ;;
esac
```

case compares one value against several patterns. Each ;; ends a branch. * catches anything not matched. esac closes the statement.

Stage F: Tool check function

```
check_tool() {
    if ! command -v "$1" >/dev/null 2>&1; then
        echo "Error: required tool '$1' is not installed or not in PATH."
        exit 1
    fi
}
```

Stage G: Final case logic

```
case "$choice" in
    1)
        check_tool whatweb
        echo "[+] Running WhatWeb against $target"
        whatweb "http://$target"
        ;;
```

```
    2)
        check_tool nmap
        echo "[+] Running Nmap service detection against $target"
        nmap -sV "$target"
        ;;
    3)
        check_tool dirb
        echo "[+] Running DIRB against $target"
        dirb "http://$target"
        ;;
    4)
        echo "Exiting. No scan was run."
        exit 0
        ;;
    *)
        echo "Error: invalid menu choice."
        exit 1
        ;;
esac
```


```
# Final case logic
case "$choice" in
    1)
        check_tool whatweb
        echo "[+] Running WhatWeb against $target"
        whatweb "http://$target"
        ;;
    2)
        check_tool nmap
        echo "[+] Running Nmap service detection against $target"
        nmap -sV "$target"
        ;;
    3)
        check_tool dirb
        echo "[+] Running DIRB against $target"
        dirb "http://$target"
        ;;
    4)
        echo "Exiting. No scan was run."
        exit 0
        ;;
    *)
        echo "Error: invalid menu choice."
        exit 1
        ;;
esac
```

Part 4 - Completed Script

```
#!/usr/bin/env bash

echo "ICDFA Beginner Reconnaissance Tool"
echo "----------------------------------"
echo "Use only against authorised lab targets."
echo

read -rp "Enter authorised target IP address or domain: " target

if [[ -z "$target" ]]; then
    echo "Error: no target was entered."
    exit 1
fi

if [[ "$target" =~ [[:space:]] ]]; then
```

```
    echo "Error: target must not contain spaces."
    exit 1
fi

check_tool() {
    if ! command -v "$1" >/dev/null 2>&1; then
        echo "Error: required tool '$1' is not installed or not in PATH."
        exit 1
    fi
}

echo "Select a reconnaissance tool:"
echo "1) WhatWeb"
echo "2) Nmap"
echo "3) DIRB"
echo "4) Exit"
read -rp "Enter your choice [1-4]: " choice

case "$choice" in
    1)
        check_tool whatweb
        echo "[+] Running WhatWeb against $target"
        whatweb "http://$target"
        ;;
    2)
        check_tool nmap
        echo "[+] Running Nmap service detection against $target"
        nmap -sV "$target"
        ;;
    3)
        check_tool dirb
        echo "[+] Running DIRB against $target"
        dirb "http://$target"
        ;;
    4)
        echo "Exiting. No scan was run."
        exit 0
        ;;
    *)
        echo "Error: invalid menu choice."
        exit 1
        ;;
esac
```

Part 5 - Save, Make Executable and Run

Step 4: Save Nano file

Objective: Save your code.

```
Ctrl+O → Enter → Ctrl+X
```

What this means: Writes the file and exits Nano.

What you should expect: Terminal returns.

Record in your report: Run cat recon_tool.sh and screenshot your own code.

```
┌──(kali㉿cyberwiz)-[~]
└─$ cat recon_tool.sh
#!/usr/bin/env bash

echo "ICDFA Beginner Reconnaissance Tool"
echo "================================="
echo "             DISCLAIMER          "
echo "================================="
echo "Use only against authorised lab targets."

# Read the target
echo
read -rp "Enter authorised target IP address or domain: " target

echo "Target entered: $target"

# Validate empty input
if [[ -z "$target" ]]; then
    echo "Error: no target was entered."
    exit 1
fi

# Menu
echo
echo "Select a reconnaissance tool:"
```

```
echo "1) WhatWeb"
echo "2) Nmap"
echo "3) DIRB"
echo "4) Exit"
read -rp "Enter your choice [1-4]: " choice

# Understand case
case "$choice" in
    1) echo "Option 1" ;;
    2) echo "Option 2" ;;
    *) echo "Invalid option" ;;
esac

# Tool check function
check_tool() {
    if ! command -v "$1" >/dev/null 2>&1; then
        echo "Error: required tool '$1' is not installed or not in PATH."
        exit 1
    fi
}

# Final case logic
case "$choice" in
    1)
        check_tool whatweb
```

```
        echo "[+] Running WhatWeb against $target"
        whatweb "http://$target"
        ;;
    2)
        check_tool nmap
        echo "[+] Running Nmap service detection against $target"
        nmap -sV "$target"
        ;;
    3)
        check_tool dirb
        echo "[+] Running DIRB against $target"
        dirb "http://$target"
        ;;
    4)
        echo "Exiting. No scan was run."
        exit 0
        ;;
    *)
        echo "Error: invalid menu choice."
        exit 1
        ;;
esac
```

Step 5: Make executable

Objective: Add execute permission.

```
chmod +x recon_tool.sh
ls -l recon_tool.sh
```

What this means: +x adds executable permission.

What you should expect: The permission string includes x.

Record in your report: Capture evidence.

Step 6: Run directly

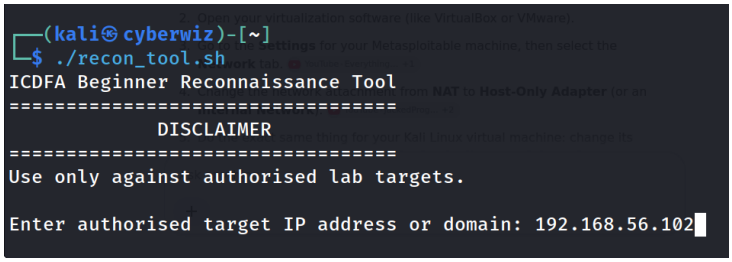
Objective: Start the program using the shebang.

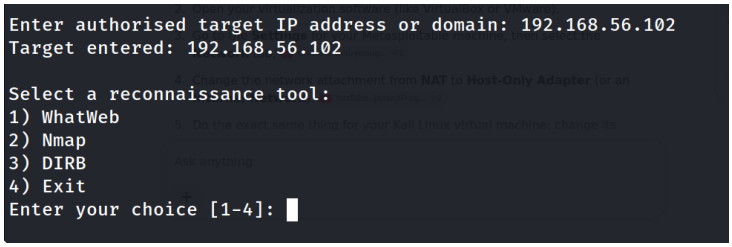
```
./recon_tool.sh
```

What this means: ./ runs a file from the current directory.

What you should expect: Target prompt and menu appear.

Record in your report: Screenshot target prompt and menu.





Part 6 - Mandatory Tool Tests

Test 1: WhatWeb

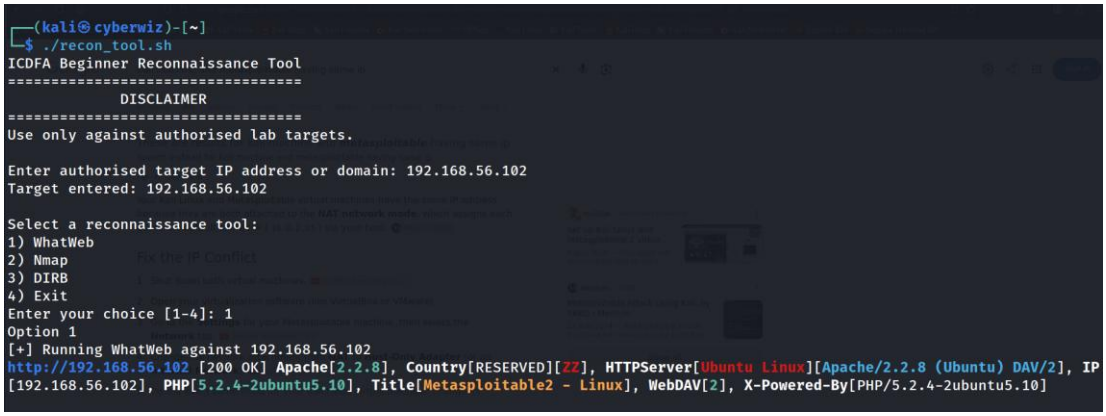
Objective: Prove option 1 works.

```
./recon_tool.sh
```

What this means: Enter the authorised web target and choose 1.

What you should expect: WhatWeb output in terminal.

Record in your report: Screenshot successful WhatWeb execution.



Test 2: Nmap

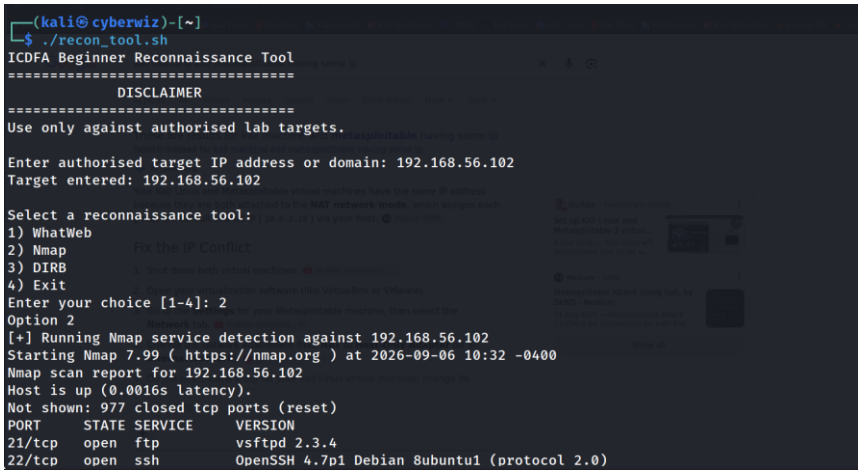
Objective: Prove option 2 works.

```
./recon_tool.sh
```

What this means: Enter the same authorised target and choose 2.

What you should expect: Nmap -sV output.

Record in your report: Screenshot successful Nmap execution.



```
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login
514/tcp  open  shell       Netkit rshd
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
```

```
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:85:97:26 (Oracle VirtualBox virtual NIC)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.38 seconds
```

Test 3: DIRB

Objective: Test the third option.

```
./recon_tool.sh
```

What this means: Enter the authorised target and choose 3.

What you should expect: DIRB path discovery output.

Record in your report: Screenshot if required; at least two tools are mandatory.

```
┌──(kali㉿cyberwiz)-[~]
└─$ ./recon_tool.sh
ICDFA Beginner Reconnaissance Tool
==================================
            DISCLAIMER
==================================
Use only against authorised lab targets.

Enter authorised target IP address or domain: 192.168.56.102
Target entered: 192.168.56.102

Select a reconnaissance tool:
1) WhatWeb
2) Nmap
3) DIRB
4) Exit
Enter your choice [1-4]: 3
Invalid option
[+] Running DIRB against 192.168.56.102

-----------------
DIRB v2.22
By The Dark Raver
-----------------
```

```
START_TIME: Sun Sep  6 10:36:56 2026
URL_BASE: http://192.168.56.102/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----------------

GENERATED WORDS: 4612

---- Scanning URL: http://192.168.56.102/ ----
+ http://192.168.56.102/cgi-bin/ (CODE:403|SIZE:295)
==> DIRECTORY: http://192.168.56.102/dav/
+ http://192.168.56.102/index (CODE:200|SIZE:891)
+ http://192.168.56.102/index.php (CODE:200|SIZE:891)
+ http://192.168.56.102/phpinfo (CODE:200|SIZE:48092)
+ http://192.168.56.102/phpinfo.php (CODE:200|SIZE:48104)
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/
+ http://192.168.56.102/server-status (CODE:403|SIZE:300)
==> DIRECTORY: http://192.168.56.102/test/
==> DIRECTORY: http://192.168.56.102/twiki/

---- Entering directory: http://192.168.56.102/dav/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/phpMyAdmin/ ----
+ http://192.168.56.102/phpMyAdmin/calendar (CODE:200|SIZE:4145)
```

```
+ http://192.168.56.102/phpMyAdmin/changelog (CODE:200|SIZE:74593)
+ http://192.168.56.102/phpMyAdmin/ChangeLog (CODE:200|SIZE:40540)
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/contrib/
+ http://192.168.56.102/phpMyAdmin/docs (CODE:200|SIZE:4583)
+ http://192.168.56.102/phpMyAdmin/error (CODE:200|SIZE:1063)
+ http://192.168.56.102/phpMyAdmin/export (CODE:200|SIZE:4145)
+ http://192.168.56.102/phpMyAdmin/favicon.ico (CODE:200|SIZE:18902)
+ http://192.168.56.102/phpMyAdmin/import (CODE:200|SIZE:4145)
+ http://192.168.56.102/phpMyAdmin/index (CODE:200|SIZE:4145)
+ http://192.168.56.102/phpMyAdmin/index.php (CODE:200|SIZE:4145)
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/js/
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/lang/
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/libraries/
+ http://192.168.56.102/phpMyAdmin/license (CODE:200|SIZE:18011)
+ http://192.168.56.102/phpMyAdmin/LICENSE (CODE:200|SIZE:18011)
+ http://192.168.56.102/phpMyAdmin/main (CODE:200|SIZE:4227)
+ http://192.168.56.102/phpMyAdmin/navigation (CODE:200|SIZE:4145)
+ http://192.168.56.102/phpMyAdmin/phpinfo (CODE:200|SIZE:0)
+ http://192.168.56.102/phpMyAdmin/phpinfo.php (CODE:200|SIZE:0)
+ http://192.168.56.102/phpMyAdmin/phpmyadmin (CODE:200|SIZE:21389)
+ http://192.168.56.102/phpMyAdmin/print (CODE:200|SIZE:1063)
+ http://192.168.56.102/phpMyAdmin/readme (CODE:200|SIZE:2624)
+ http://192.168.56.102/phpMyAdmin/README (CODE:200|SIZE:2624)
+ http://192.168.56.102/phpMyAdmin/robots (CODE:200|SIZE:26)
+ http://192.168.56.102/phpMyAdmin/robots.txt (CODE:200|SIZE:26)
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/scripts/
```

```
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/setup/
+ http://192.168.56.102/phpMyAdmin/sql (CODE:200|SIZE:4145)
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/test/
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/themes/
+ http://192.168.56.102/phpMyAdmin/TODO (CODE:200|SIZE:235)
+ http://192.168.56.102/phpMyAdmin/webapp (CODE:200|SIZE:6902)

---- Entering directory: http://192.168.56.102/test/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/twiki/ ----
==> DIRECTORY: http://192.168.56.102/twiki/bin/
+ http://192.168.56.102/twiki/data (CODE:403|SIZE:297)
+ http://192.168.56.102/twiki/index (CODE:200|SIZE:782)
+ http://192.168.56.102/twiki/index.html (CODE:200|SIZE:782)
==> DIRECTORY: http://192.168.56.102/twiki/lib/
+ http://192.168.56.102/twiki/license (CODE:200|SIZE:19440)
==> DIRECTORY: http://192.168.56.102/twiki/pub/
+ http://192.168.56.102/twiki/readme (CODE:200|SIZE:4334)
+ http://192.168.56.102/twiki/templates (CODE:403|SIZE:302)

---- Entering directory: http://192.168.56.102/phpMyAdmin/contrib/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)
```

```
---- Entering directory: http://192.168.56.102/phpMyAdmin/js/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/phpMyAdmin/lang/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/phpMyAdmin/libraries/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/phpMyAdmin/scripts/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/phpMyAdmin/setup/ ----
+ http://192.168.56.102/phpMyAdmin/setup/config (CODE:303|SIZE:1370)
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/setup/frames/
+ http://192.168.56.102/phpMyAdmin/setup/index (CODE:200|SIZE:8618)
+ http://192.168.56.102/phpMyAdmin/setup/index.php (CODE:200|SIZE:8626)
==> DIRECTORY: http://192.168.56.102/phpMyAdmin/setup/lib/
+ http://192.168.56.102/phpMyAdmin/setup/scripts (CODE:200|SIZE:21967)
+ http://192.168.56.102/phpMyAdmin/setup/styles (CODE:200|SIZE:6218)
```

```
---- Entering directory: http://192.168.56.102/phpMyAdmin/test/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/phpMyAdmin/themes/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/twiki/bin/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/twiki/lib/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/twiki/pub/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.56.102/phpMyAdmin/setup/frames/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)
```

```
---- Entering directory: http://192.168.56.102/phpMyAdmin/setup/lib/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

-----------------
END_TIME: Sun Sep  6 10:38:01 2026
DOWNLOADED: 18448 - FOUND: 42
```

Part 7 - Optional Improvement: Save Output

```
mkdir -p results
nmap -sV "$target" | tee "results/nmap-$target.txt"
whatweb "http://$target" | tee "results/whatweb-$target.txt"
dirb "http://$target" | tee "results/dirb-$target.txt"
```

tee displays output and writes the same output to a file. This is optional after the mandatory version works.

Part 9 - Explanation Guide

1. Purpose of the script.

A script is used to automate the execution of a series of commands or tasks.

2. How read stores the target.

read stores the target (user's input) in the specified variable.

3. How the menu stores choice.

The menu stores the user's selection in a variable (e.g., choice) using the read command.

4. How case selects a tool.

case selects a tool by matching the user's choice against predefined options and execute the corresponding command block afterwards.

5. Why no target is hardcoded.

No target is hardcoded because the script gets the target from user input instead of embedding a fixed value in the code. A hardcoded target would require that the code will be edited each time there is a new target which is not the essence for scripting.

6. Why chmod +x is required.

chmod +x adds execute permissions to a specified file, without adding execute permission to a file it cannot be executed.

7. Why authorisation matters.

Authorization matters because it ensures that only permitted users can access or perform specific actions on a system, file, or resource.

Part 10 - Understanding Questions

1. What does the shebang do?

Shebang is the first line of a script that tells the operating system which interpreter should execute the script e.g bash.

2. What does read -rp do?

read -rp combines two options of the Bash read command:

- -r → Prevents backslashes (\) from being treated as escape characters (special characters that begin with a backslash).
- -p → Displays a prompt before reading user input.

3. Why quote "$target"?

"$target" is quoted to prevent problems caused by spaces, empty values, and special characters in the variable.

4. What does -z test?

-z tests whether a string is empty (zero length).

5. What is the purpose of exit 1?

exit 1 is used in a shell script terminate a script and indicate that an error occurred.

6. Why is case suitable for menus?

Case statement is suitable for menus because it allows a program to easily handle multiple user choices in a clear and organized way.

7. What does ;; mean?

Each ;; ends a branch.

8. What does * mean in the case statement?

* catches anything not matched.

9. Why does WhatWeb/DIRB use http://$target?

WhatWeb/DIRB use http://$target because they send HTTP request.

10. What does nmap -sV do?

-sV sends service probes and tries to identify products/versions.

11. What does command -v check?

checks whether a command is available in the current shell environment and shows error prompt on how it would be resolved.

12. What does chmod +x do?

chmod +x adds execute permissions to a specified file.

13. What does ./ mean?

By default, Linux/Unix shells search for executable commands only in directories listed in the PATH environment variable.

So using ./ explicitly telling Bash where the file is located.

. is Current directory.

./ takes you into the current directory specified.

14. What does tee do?

`tee` is a Linux command that:

1. Displays output on the screen.
2. Writes the same output to a file.

This allows viewing and saving output simultaneously.

15. Which requirement prevents hardcoding?

In software development, the requirement that prevents **hardcoding** is typically a **configuration requirement** or **parameterization requirement**, Such as Passwords, File paths, URLs must be stored in configuration files, environment variables, databases, or user inputs rather than being embedded directly in the source code.

Conclusion

After completing Lab 3, I gained knowledge and practical application for:

Bash Scripting automation practical using Nmap, WhatWeb and DIRB to build a tool for reconnaissance. That Prompts for a target IP/domain, display a menu: WhatWeb, Nmap, DIRB, runs the selected tool against the supplied target and displays results.